Lab-6

1. Use MongoDB to implement the following DB operations

   1. Create a database called ‘vehicles’ and *write* a MongoDB query to select database as “vehicles”.

      Code:

      use vehicles

      Output:

      ```
      test> use vehicles
      switched to db vehicles
      ```

   2. Write a MongoDB query to display all the databases.

      Code:

      show dbs

      Output:

      ```
      vehicles> show dbs
      admin      40.00 KiB
      config    104.00 KiB
      local      40.00 KiB
      vehicles   80.00 KiB
      ```

   3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called ‘four_wheelers’.

      Code:

      db.createCollection("two_wheelers",{capped:true,size:50000})
      db.createCollection("four_wheelers")

      Output:

      ```
      vehicles> db.createCollection("two_wheelers",{capped:true,size:50000})
      { ok: 1 }
      vehicles> db.createCollection("four_wheelers")
      { ok: 1 }
      ```

   4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Code:

```
db.two_wheelers.insertMany([
{bike_name:"Jupiter",model:"gearless",category:"110cc",colors_available:["white","blue"],manufacturer:"TVS",performance:8,timestamp:"2022",price:85000},
{bike_name:"Splendor",model:"gear",category:"100cc",colors_available:["black","red"],manufacturer:"Hero",performance:7,timestamp:"2021",price:75000},
{bike_name:"R15",model:"gear",category:"155cc",colors_available:["blue","black"],manufacturer:"Yamaha",performance:9,timestamp:"2023",price:180000},
{bike_name:"Duke",model:"gear",category:"200cc",colors_available:["orange","black"],manufacturer:"KTM",performance:9,timestamp:"2022",price:210000},
{bike_name:"Access",model:"gearless",category:"125cc",colors_available:["grey","white"],manufacturer:"Suzuki",performance:8,timestamp:"2020",price:90000}
])
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b175e4079a7f48047c2907'),
    '1': ObjectId('69b175e4079a7f48047c2908'),
    '2': ObjectId('69b175e4079a7f48047c2909'),
    '3': ObjectId('69b175e4079a7f48047c290a'),
    '4': ObjectId('69b175e4079a7f48047c290b')
  }
}
```

5. Add 5 four-wheeler details to the collection named 'four_wheelers'. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Code:

```
db.four_wheelers.insertMany([
{vehicle_name:"Baleno",model:"own",category:"car",variants:["zeta","petrol"],manufacturer:"Maruti",performance:8,timestamp:"2021",price:850000},
{vehicle_name:"Fortuner",model:"own",category:"car",variants:["diesel"],manufacturer:"Toyota",performance:9,timestamp:"2023",price:4200000},
{vehicle_name:"Scorpio",model:"own",category:"car",variants:["diesel"],manufacturer:"Mahindra",performance:9,timestamp:"2022",price:1800000},
```

{vehicle_name:"VolvoBus",model:"commercial",category:"bus",variants:["diesel"],manufacturer:"Volvo",performance:8,timestamp:"2020",price:6000000},
{vehicle_name:"EicherTruck",model:"commercial",category:"heavy truck",variants:["diesel"],manufacturer:"Eicher",performance:8,timestamp:"2022",price:3200000}
])

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b175ec079a7f48047c290c'),
    '1': ObjectId('69b175ec079a7f48047c290d'),
    '2': ObjectId('69b175ec079a7f48047c290e'),
    '3': ObjectId('69b175ec079a7f48047c290f'),
    '4': ObjectId('69b175ec079a7f48047c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

Code:

db.two_wheelers.find()
db.four_wheelers.find()

Output:

```
[
  {
    _id: ObjectId('69b175e4079a7f48047c2907'),
    bike_name: 'Jupiter',
    model: 'gearless',
    category: '110cc',
    colors_available: [ 'white', 'blue' ],
    manufacturer: 'TVS',
    performance: 8,
    timestamp: '2022',
    price: 85000
  },
  {
    _id: ObjectId('69b175e4079a7f48047c2908'),
    bike_name: 'Splendor',
    model: 'gear',
    category: '100cc',
    colors_available: [ 'black', 'red' ],
    manufacturer: 'Hero',
    performance: 7,
    timestamp: '2021',
    price: 75000
  },
  {
    _id: ObjectId('69b175e4079a7f48047c2909'),
    bike_name: 'R15',
    model: 'gear',
    category: '155cc',
    colors_available: [ 'blue', 'black' ],
    manufacturer: 'Yamaha',
    performance: 9,
    timestamp: '2023',
    price: 180000
  },
  {
    _id: ObjectId('69b175e4079a7f48047c290a'),
    bike_name: 'Duke',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'orange', 'black' ],
    manufacturer: 'KTM',
    performance: 9,
    timestamp: '2022',
    price: 210000
  },
```

```
  {
    _id: ObjectId('69b175e4079a7f48047c290b'),
    bike_name: 'Access',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'grey', 'white' ],
    manufacturer: 'Suzuki',
    performance: 8,
    timestamp: '2020',
    price: 90000
  }
]
vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69b175ec079a7f48047c290c'),
    vehicle_name: 'Baleno',
    model: 'own',
    category: 'car',
    variants: [ 'zeta', 'petrol' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: '2021',
    price: 850000
  },
  {
    _id: ObjectId('69b175ec079a7f48047c290d'),
    vehicle_name: 'Fortuner',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: '2023',
    price: 4200000
  },
```

```
  {
    _id: ObjectId('69b175ec079a7f48047c290e'),
    vehicle_name: 'Scorpio',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 9,
    timestamp: '2022',
    price: 1800000
  },
  {
    _id: ObjectId('69b175ec079a7f48047c290f'),
    vehicle_name: 'VolvoBus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Volvo',
    performance: 8,
    timestamp: '2020',
    price: 6000000
  },
  {
    _id: ObjectId('69b175ec079a7f48047c2910'),
    vehicle_name: 'EicherTruck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 8,
    timestamp: '2022',
    price: 3200000
  }
]
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

Code:

db.two_wheelers.find({}, {bike_name:1,price:1,_id:0})
db.four_wheelers.find({}, {vehicle_name:1,price:1,_id:0})

Output:

```
vehicles> db.two_wheelers.find({}, {bike_name:1,price:1,_id:0})
[
  { bike_name: 'Jupiter', price: 85000 },
  { bike_name: 'Splendor', price: 75000 },
  { bike_name: 'R15', price: 180000 },
  { bike_name: 'Duke', price: 210000 },
  { bike_name: 'Access', price: 90000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1,price:1,_id:0})
[
  { vehicle_name: 'Baleno', price: 850000 },
  { vehicle_name: 'Fortuner', price: 4200000 },
  { vehicle_name: 'Scorpio', price: 1800000 },
  { vehicle_name: 'VolvoBus', price: 6000000 },
  { vehicle_name: 'EicherTruck', price: 3200000 }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

   Code:

   db.two_wheelers.find({manufacturer:"Honda"})

   Output:

   No output, since there is no vehicle named honda.

9. Write a MongoDB query to display four_wheelers available in diesel variants

   Code:

   db.four_wheelers.find({variants:"diesel"})

   Output:

```
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b175ec079a7f48047c290d'),
    vehicle_name: 'Fortuner',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: '2023',
    price: 4200000
  },
  {
    _id: ObjectId('69b175ec079a7f48047c290e'),
    vehicle_name: 'Scorpio',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 9,
    timestamp: '2022',
    price: 1800000
  },
  {
    _id: ObjectId('69b175ec079a7f48047c290f'),
    vehicle_name: 'VolvoBus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Volvo',
    performance: 8,
    timestamp: '2020',
    price: 6000000
  },
  {
    _id: ObjectId('69b175ec079a7f48047c2910'),
    vehicle_name: 'EicherTruck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 8,
    timestamp: '2022',
    price: 3200000
  }
]
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

Code:

db.two_wheelers.find({performance:{$gt:5}},{bike_name:1,category:1,manufacturer:1,_id:0})
db.four_wheelers.find({performance:{$gt:5}},{vehicle_name:1,category:1,manufacturer:1,_id:0})

Output:

```
vehicles> db.two_wheelers.find({performance:{$gt:5}},{bike_name:1,category:1,manufacturer:1,_id:0})
[
  { bike_name: 'Jupiter', category: '110cc', manufacturer: 'TVS' },
  { bike_name: 'Splendor', category: '100cc', manufacturer: 'Hero' },
  { bike_name: 'R15', category: '155cc', manufacturer: 'Yamaha' },
  { bike_name: 'Duke', category: '200cc', manufacturer: 'KTM' },
  { bike_name: 'Access', category: '125cc', manufacturer: 'Suzuki' }
]
vehicles> db.four_wheelers.find({performance:{$gt:5}},{vehicle_name:1,categocategory:1,manufacturer:1,_id:0})
[
  { vehicle_name: 'Baleno', category: 'car', manufacturer: 'Maruti' },
  { vehicle_name: 'Fortuner', category: 'car', manufacturer: 'Toyota' },
  { vehicle_name: 'Scorpio', category: 'car', manufacturer: 'Mahindra' },
  { vehicle_name: 'VolvoBus', category: 'bus', manufacturer: 'Volvo' },
  {
    vehicle_name: 'EicherTruck',
    category: 'heavy truck',
    manufacturer: 'Eicher'
  }
]
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and write a MongoDB query to select database as ‘animal’.

Code:

use animal

Output:

```
test> use animal
switched to db animal
```

2. Write a MongoDB query to display all the databases.

Code:

show dbs

Output:

```
animal> show dbs
admin      40.00 KiB
config     36.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

Code:

db.createCollection("wild_animals",{capped:true,size:50000})
db.createCollection("domestic_animals")

Output:

```
animal> db.createCollection("wild_animals",{capped:true,size:50000})
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Code:

```
db.wild_animals.insertMany([
{
animal_name:"Leopard",
nature:"harm",
favorite_foods:["meat","goat"],
care_taker_name:"Mahesh",
life_span:17,
timestamp:"2023",
expenses:52000
},
{
animal_name:"Wolf",
nature:"harm",
favorite_foods:["meat","sheep"],
care_taker_name:"Ramesh",
life_span:14,
timestamp:"2022",
expenses:43000
},
{
animal_name:"Giraffe",
nature:"harmless",
favorite_foods:["leaves","plants"],
care_taker_name:"Sanjay",
life_span:25,
timestamp:"2021",
expenses:70000
},
{
animal_name:"Fox",
nature:"harm",
favorite_foods:["chicken","fish"],
care_taker_name:"Mahesh",
life_span:10,
timestamp:"2020",
expenses:30000
},
{
animal_name:"Zebra",
nature:"harmless",
favorite_foods:["grass","plants"],
```

```
care_taker_name:"Anil",
life_span:22,
timestamp:"2022",
expenses:48000
}
])
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b18db1bf30e723e17c2907'),
    '1': ObjectId('69b18db1bf30e723e17c2908'),
    '2': ObjectId('69b18db1bf30e723e17c2909'),
    '3': ObjectId('69b18db1bf30e723e17c290a'),
    '4': ObjectId('69b18db1bf30e723e17c290b')
  }
}
```

5.Add 5 domestic-animal details to the collection named 'domestic_animals'. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Code:

```
db.domestic_animals.insertMany([
{
animal_name:"Horse",
gender:"male",
favorite_foods:["grass","grain"],
animal_petname:"Thunder",
life_span:25,
timestamp:"2023",
expenses:20000
},
{
animal_name:"Parrot",
gender:"female",
favorite_foods:["seeds","fruits"],
animal_petname:"Mithu",
life_span:12,
```

timestamp:"2022",
expenses:4000
},
{
animal_name:"Buffalo",
gender:"female",
favorite_foods:["grass","fodder"],
animal_petname:"Lakshmi",
life_span:18,
timestamp:"2021",
expenses:16000
},
{
animal_name:"Duck",
gender:"male",
favorite_foods:["grains","insects"],
animal_petname:"Donald",
life_span:9,
timestamp:"2020",
expenses:3000
},
{
animal_name:"Hamster",
gender:"female",
favorite_foods:["seeds","nuts"],
animal_petname:"Snowy",
life_span:4,
timestamp:"2022",
expenses:2000
}
])

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b18dbebf30e723e17c290c'),
    '1': ObjectId('69b18dbebf30e723e17c290d'),
    '2': ObjectId('69b18dbebf30e723e17c290e'),
    '3': ObjectId('69b18dbebf30e723e17c290f'),
    '4': ObjectId('69b18dbebf30e723e17c2910')
  }
}
```

6.Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

Code:

db.wild_animals.find()
db.domestic_animals.find()

Output:

```
animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69b18db1bf30e723e17c2907'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'meat', 'goat' ],
    care_taker_name: 'Mahesh',
    life_span: 17,
    timestamp: '2023',
    expenses: 52000
  },
  {
    _id: ObjectId('69b18db1bf30e723e17c2908'),
    animal_name: 'Wolf',
    nature: 'harm',
    favorite_foods: [ 'meat', 'sheep' ],
    care_taker_name: 'Ramesh',
    life_span: 14,
    timestamp: '2022',
    expenses: 43000
  },
  {
    _id: ObjectId('69b18db1bf30e723e17c2909'),
    animal_name: 'Giraffe',
    nature: 'harmless',
    favorite_foods: [ 'leaves', 'plants' ],
    care_taker_name: 'Sanjay',
    life_span: 25,
    timestamp: '2021',
    expenses: 70000
  },
  {
    _id: ObjectId('69b18db1bf30e723e17c290a'),
    animal_name: 'Fox',
    nature: 'harm',
    favorite_foods: [ 'chicken', 'fish' ],
    care_taker_name: 'Mahesh',
    life_span: 10,
    timestamp: '2020',
    expenses: 30000
  },
```

```
  {
    _id: ObjectId('69b18db1bf30e723e17c290b'),
    animal_name: 'Zebra',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'plants' ],
    care_taker_name: 'Anil',
    life_span: 22,
    timestamp: '2022',
    expenses: 48000
  }
]
animal> db.domestic_animals.find()
[
  {
    _id: ObjectId('69b18dbebf30e723e17c290c'),
    animal_name: 'Horse',
    gender: 'male',
    favorite_foods: [ 'grass', 'grain' ],
    animal_petname: 'Thunder',
    life_span: 25,
    timestamp: '2023',
    expenses: 20000
  },
  {
    _id: ObjectId('69b18dbebf30e723e17c290d'),
    animal_name: 'Parrot',
    gender: 'female',
    favorite_foods: [ 'seeds', 'fruits' ],
    animal_petname: 'Mithu',
    life_span: 12,
    timestamp: '2022',
    expenses: 4000
  },
```

```
  {
    _id: ObjectId('69b18dbebf30e723e17c290e'),
    animal_name: 'Buffalo',
    gender: 'female',
    favorite_foods: [ 'grass', 'fodder' ],
    animal_petname: 'Lakshmi',
    life_span: 18,
    timestamp: '2021',
    expenses: 16000
  },
  {
    _id: ObjectId('69b18dbebf30e723e17c290f'),
    animal_name: 'Duck',
    gender: 'male',
    favorite_foods: [ 'grains', 'insects' ],
    animal_petname: 'Donald',
    life_span: 9,
    timestamp: '2020',
    expenses: 3000
  },
  {
    _id: ObjectId('69b18dbebf30e723e17c2910'),
    animal_name: 'Hamster',
    gender: 'female',
    favorite_foods: [ 'seeds', 'nuts' ],
    animal_petname: 'Snowy',
    life_span: 4,
    timestamp: '2022',
    expenses: 2000
  }
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

Code:

db.wild_animals.find({}, {animal_name:1,expenses:1,_id:0})
db.domestic_animals.find({}, {animal_name:1,expenses:1,_id:0})

Output:

```
animal> db.wild_animals.find({}, {animal_name:1,expenses:1,_id:0})
[
  { animal_name: 'Leopard', expenses: 52000 },
  { animal_name: 'Wolf', expenses: 43000 },
  { animal_name: 'Giraffe', expenses: 70000 },
  { animal_name: 'Fox', expenses: 30000 },
  { animal_name: 'Zebra', expenses: 48000 }
]
animal> db.domestic_animals.find({}, {animal_name:1,expenses:1,_id:0})
[
  { animal_name: 'Horse', expenses: 20000 },
  { animal_name: 'Parrot', expenses: 4000 },
  { animal_name: 'Buffalo', expenses: 16000 },
  { animal_name: 'Duck', expenses: 3000 },
  { animal_name: 'Hamster', expenses: 2000 }
]
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

Code:

db.domestic_animals.find({life_span:10})

Output:

No output, since there is no animal whose life span is 10 years.

9. Write a MongoDB query to display wild_animals available under a particular care_taker

Code:

db.wild_animals.find({care_taker_name:"Ravi"})

Output:

No output, since there is no care taker named “Ravi”.

10.Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

Code:

db.wild_animals.find({life_span:{$gt:5}},
{animal_name:1,favorite_foods:1,expenses:1,_id:0})

db.domestic_animals.find({life_span:{$gt:5}},
{animal_name:1,favorite_foods:1,expenses:1,_id:0})

Output:

```
[
  {
    animal_name: 'Leopard',
    favorite_foods: [ 'meat', 'goat' ],
    expenses: 52000
  },
  {
    animal_name: 'Wolf',
    favorite_foods: [ 'meat', 'sheep' ],
    expenses: 43000
  },
  {
    animal_name: 'Giraffe',
    favorite_foods: [ 'leaves', 'plants' ],
    expenses: 70000
  },
  {
    animal_name: 'Fox',
    favorite_foods: [ 'chicken', 'fish' ],
    expenses: 30000
  },
  {
    animal_name: 'Zebra',
    favorite_foods: [ 'grass', 'plants' ],
    expenses: 48000
  }
]
```

```
[
  {
    animal_name: 'Horse',
    favorite_foods: [ 'grass', 'grain' ],
    expenses: 20000
  },
  {
    animal_name: 'Parrot',
    favorite_foods: [ 'seeds', 'fruits' ],
    expenses: 4000
  },
  {
    animal_name: 'Buffalo',
    favorite_foods: [ 'grass', 'fodder' ],
    expenses: 16000
  },
  {
    animal_name: 'Duck',
    favorite_foods: [ 'grains', 'insects' ],
    expenses: 3000
  }
]
```